ENGINEERING STUDY · 2026.09

cpp-csharp-lsp-lite

폐쇄망에서 코드 수정과 검증을 연결하기

사내 LLM · pi 실행 환경 · C++ / C# 언어 서버를 결합한
코딩 에이전트의 설계와 검증

문제 배경 | 설계 선택 | LSP | 검증 | 설치 부록

Windows x64 **사내 모델 API** **오프라인 배포** **pi 0.87.1**

핵심 질문: AI가 고친 코드를 어떻게 확인할까?

폐쇄망에서도 프로젝트를 탐색하고 수정한 뒤,
언어 분석 결과를 다음 판단에 돌려줄 수 있을까?

01

업무 제약

사내 API · Windows
C++ / C# 프로젝트

→

02

실행 환경

pi에 탐색·편집·분석
도구를 연결

→

03

검증과 도입

기능 검증 후
실제 업무 효과 측정

발표 범위: 문제 → 대안의 한계 → 설계 → 구현 → 검증 → 남은 과제

도입 조건은 모델 연결보다 넓다

네트워크와 배포

외부 다운로드가 제한된 환경.
실행 파일·의존성·분석 리소스를
반입 전에 준비해야 한다.

프로젝트 맥락

C++의 헤더·매크로·컴파일 옵션,
C#의 솔루션·SDK·패키지가
분석 결과를 좌우한다.

기존 업무와 연결

Windows 개발 PC에서 사용하고,
기존 IDE·빌드·테스트 절차와
함께 운영할 수 있어야 한다.

대상 환경 외부 인터넷 없이 배포·실행하되, 모델 추론에는 사내 API 연결을 사용한다.

필요한 것은 모델 접속, 프로젝트 맥락, 배포 재현성을 함께 갖춘 작업 환경이다.

대화·검색·빌드만으로는 피드백이 끊긴다

비교할 접근	제공하는 것	이 과제에서 남는 연결 작업
웹 대화 + 코드 복사	설명·수정안 생성	관련 파일 선택, 패치 적용, 오류 전달을 사람이 수행
파일 읽기 + 문자열 검색	이름·텍스트 기반의 빠른 탐색	동명이인 심볼, 타입, 실제 참조 관계를 추가 확인
IDE에서 수동 확인	편집 위치의 진단·탐색	IDE의 분석 결과를 에이전트의 다음 판단에 전달
수정 후 전체 빌드·테스트	통합 상태와 실행 동작 검증	작은 수정 단계에서도 활용할 언어 분석 피드백

설계 목표: 탐색 → 수정 → 진단 → 재수정 사이의 전달 작업을 줄인다.

업무 제약을 네 가지 설계 기준으로 바꾼다

요구사항	설계 대응	확인 기준
사내 모델 사용	OpenAI-compatible API 설정	등록된 provider/model로 도구 호출·응답
폐쇄망 재배포	버전·해시 고정, 런타임과 의존성 동봉	외부 설치 없이 재배치한 ZIP에서 실행
프로젝트 기반 피드백	clangd / csharp-ls를 LSP로 연결	정의·참조·타입·진단 결과 조회
변경 후 확인	edit/write 결과에 진단 첨부	오류 발생과 수정 후 진단 변화 확인

도구 선택은 이 요구사항을 충족하기 위한 설계 판단이다.

Harness는 모델이 일하는 실행 환경이다

01

모델

맥락을 읽고
다음 행동을 선택

→

02

pi harness

세션·도구 호출·결과 전달
확장 이벤트 처리

→

03

개발 환경

파일·명령 실행
언어 서버·빌드·테스트

모델의 역할

수정 방안을 제안하고 필요한 도구를 선택한다.

Harness의 역할

선택한 도구를 실행하고 관찰 결과를 모델에 되돌린다.

모델을 바꾸는 문제와, 모델에 어떤 도구·피드백을 제공할지는 분리해서 설계한다.

왜 pi를 기반으로 구성했는가?

이 과제의 기준	pi에서 활용한 특성	구현 근거
사내 API 연결	provider·baseUrl·모델을 설정으로 지정	`config/models.example.json`
도구와 피드백 확장	TypeScript 도구 등록·편집 결과 이벤트	`extensions/cpp-csharp-lsp-lite/index.ts`
Windows 배포	공식 standalone 실행 파일과 리소스 동봉	`bin/pi/`
필요 기능 추가	배포 폴더를 pi 패키지로 로드	`package.json` · `config/settings.json`

근거: 동봉 pi 0.87.1 문서 및 현재 저장소 구현. 제품 간 성능 비교 실험은 수행하지 않음.

선택의 대가: 확장 API 호환성과 Windows 동작은 배포자가 계속 검증해야 한다.

사내 추론과 로컬 분석을 하나의 작업 흐름으로

사내망 · 모델 서비스

사내 LLM API 전달받은 맥락으로 응답·도구 호출 생성 — 모델 추론

↑ 프롬프트·도구 결과 / ↓ 응답·도구 호출

Windows 개발 PC · 배포 ZIP + 대상 프로젝트

pi + extensions
세션과 편집 흐름
도구 실행·진단 첨부

↔

LSP client
언어·프로젝트 루트별 연결
stdio JSON-RPC

↔

언어 서버
C++: clangd
C#: csharp-ls + .NET

LSP: 언어 분석 기능을 호출하는 공통 규약

Language Server Protocol

개발 도구와 언어 서버가 코드의 위치·심볼·진단을 주고받는 약속

01

클라이언트

"이 위치의 정의는?"
"변경 파일의 오류는?"

→

02

LSP 메시지

파일 URI · 줄 · 문자 위치
요청 / 응답 / 알림

→

03

언어 서버

언어와 프로젝트 설정으로
분석한 결과 반환

분석을 수행하는 구현

clangd는 C/C++, csharp-ls는 C#의 언어 서버다.

이번 구성에서의 사용처

IDE가 사용하던 분석 기능을 pi 도구에서도 호출한다.

출처: Microsoft · Language Server Protocol

같은 문자열과 같은 심볼은 다르다

```
// 개념 예시: 동명 멤버 함수
struct Sensor { int read(); };
struct Buffer { int read(); };

Sensor sensor;
Buffer buffer;
sensor.read();  // Sensor::read
buffer.read();  // Buffer::read
```

개념 근거: LSP의 definition · references · hover

문자열 검색: read 후보 수집

선언·호출·주석 등 이름이 나타난 곳을 찾는다.

LSP: 위치와 타입을 기준으로 질문

sensor.read()의 정의와 참조,
타입·서명을 서버에 조회한다.

이름 검색에 프로젝트의 의미 정보를 더해, 변경 대상을 확인한다.

편집 직후 진단을 다음 행동으로 돌려준다

01
대상 확인
정의·참조·타입 조회
변경할 코드 파악

→

02
파일 수정
pi edit / write
저장된 파일 반영

→

03
진단 수집
언어 서버 분석
오류·경고 조회

→

04
결과 전달
편집 결과에 첨부
모델이 후속 수정

진단을 근거로 다시 탐색·수정 ↰

진단이 도착한 경우
오류 위치·메시지를 다음 판단의 근거로 제공한다.

응답이 지연되거나 실패한 경우
pending / 실패를 표시한다. 무응답을 오류 없음으로 처리하지 않는다.

언어 진단은 수정 중간의 피드백이며, 최종 빌드·테스트와 함께 사용한다.

분석의 품질은 프로젝트 맥락에 달려 있다

C / C++

compile_commands.json 또는 compile_flags.txt

헤더 경로 · 전처리 매크로 · 언어 표준
MSVC / Windows SDK / 외부 헤더

C#

solution / csproj와 맞는 SDK

프로젝트 참조 · 대상 프레임워크
기존 환경의 NuGet 의존성

실패 해석 헤더·패키지 누락으로 발생한 진단은 먼저 분석 환경을 점검한다.

출처: clangd · Compile commands / 저장소 README의 프로젝트 준비 조건

실행 도구는 ZIP으로 배포하고, 프로젝트별 빌드 맥락은 별도로 준비한다.

세 확장이 서로 다른 역할을 맡는다

구성요소	역할	현재 배포 설정
cpp-csharp-lsp-lite	C++·C# 정의 / 참조 / 타입 / 심볼 / 진단	기존 언어 서버 연결과 편집 후 진단 유지
pi-lens 4.3.0	구조 분석·추가 진단·심볼 탐색	자체 LSP 끔, 분석 WASM·ast-grep 동봉
pi-subagents 0.73.1	조사·구현·검토 등의 작업 위임	foreground 사용, 사내 모델 설정 필요

운영 경계 추가 린터는 준비된 도구 범위에서 사용. Windows background 서브에이전트는 실험적 지원.

pi-lens · pi-subagents standalone 지원 범위 · config 및 버전 잠금 파일

확장은 역할이 필요한 곳에 추가하고, 중복 분석과 실행 제약을 명시한다.

설치 절차를 재현 가능한 오프라인 묶음으로

01

일반망 준비

버전·해시 고정 다운로드
실행 파일·SDK·확장 조립

→

02

검증·반입

파일 manifest와 ZIP 체크섬
기존 망간자료전송 절차

→

03

방산망 실행

ZIP 압축 해제 · 사내 API 설정
setup.ps1 1회 → 어디서나 pi

배포에 포함

pi · clangd · csharp-ls · .NET SDK
확장 의존성 · Windows 네이티브 파일 · WASM

별도로 준비

config/models.json · API KEY
프로젝트별 헤더·SDK·NuGet 패키지

대상 PC: setup.ps1이 시스템 Path와 설정 폴더(config/)를 등록한다. 로그인 때 초기화되는 사용자 변수는 쓰지 않는다.

14 · 검증 결과

동작 확인과 업무 효과 입증을 구분한다

검증 항목	확인 상태	증거 / 범위
LSP 클라이언트·패키징	로컬 통과	단위 테스트 11개, TypeScript 타입 검사
세 확장 동시 로딩	로컬 통과	도구 중복 없음, pi 0.87.1 로더
실제 도구 호출·서브에이전트	로컬 통과	C++ 구조 분석·LSP·자식 응답 / 모의 모델 API
오프라인 ZIP	로컬 통과	캐시 재빌드, 10,014개 파일 해시·체크섬 확인
새 확장 포함 Windows 실행	Windows CI 통과	97b5071 · 재배치·서버 차단·standalone 호출
사내 프로젝트 생산성	미측정	실제 모델·프로젝트에서 후속 평가 필요

검증 기록: tests/ · dist/*.sha256 · 2026-09-29 로컬 확인. 새 확장 조합 Windows CI 성공 확인 (97b5071).

현재 확보한 근거는 통합 동작과 배포 재현성이다. 생산성 개선 수치는 아직 없다.

다음 실험은 “LSP가 실제로 도움이 되는가”

비교 조건	동일하게 고정할 항목
A · pi 기본 도구	동일 모델·설정, 동일 프로젝트 스냅샷과 과제, 동일 시간·토큰 예산과 빌드·테스트 기준
B · 기본 도구 + LSP	

과제

함수 변경 · 참조 수정

타입 오류 해결 · 회귀 수정

측정

완료율 · 소요 시간

수정 반복 · 토큰 · 사람 개입

판정

빌드·테스트 통과

의도한 변경인지 코드 리뷰

반복 실행과 실패 사례를 포함해 평가한 뒤, lens·subagents의 추가 효과를 따로 측정한다.

운영 도입 전에 확인할 네 가지

01 · 분석 환경 의존

SDK·헤더·빌드 옵션이 어긋나면 진단 품질이 낮아진다.

다음: 실제 C++·C# 프로젝트별 준비 절차 검증

02 · 검증 범위

LSP는 실행 동작이나 업무 요구사항 전체를 증명하지 않는다.

다음: 빌드·테스트·리뷰를 완료 기준으로 유지

03 · 확장 호환성

pi·확장 버전과 OS에 따라 동작이 달라질 수 있다.

다음: 버전 고정 + Windows 통합 회귀 검사

04 · 모델과 비용

모델 품질·문맥 한도·작업 위임이 결과와 비용에 영향을 준다.

다음: 사내 모델별 완료율·토큰·대기 시간 측정

도입은 대표 프로젝트의 작은 과제부터

01

준비

완성 ZIP + 사내 API
프로젝트 빌드 맥락

→

02

연결 확인

모델 응답·정의 탐색
진단 수신 확인

→

03

작은 수정

범위 지정 → 수정
진단 → 빌드·테스트

→

04

확대 판단

리뷰와 변경 기록
효과·실패·비용 비교

pi로 실행을 연결하고, LSP로 언어 분석을 돌려준다.
업무 효과는 실제 프로젝트에서 측정한다.

제안: 재현 가능한 배포와 짧은 피드백 흐름을 먼저 확보하고, 실측으로 확대를 결정한다.

현재 구성: setup.ps1로 한 번 설정하고 pi로 실행

```
# 배포 폴더에서 한 번 (관리자 승인)
Set-ExecutionPolicy -Scope Process Bypass
.\setup.ps1
# 이후 새 터미널의 어느 폴더에서나
pi

# 아무것도 바꾸지 않고 필수 파일만 확인
.\setup.ps1 -Check
```

setup.ps1이 등록하는 시스템 변수

Path: bin(fd·rg) · bin\pi · SVN · ast-grep

PI_CODING_AGENT_DIR = 배포 폴더\config

포함 제외

실제 config/models.json · 인증 정보

config/ 아래 세션·캐시

GitHub Release의 완성 ZIP 하나를 반입한다. pi 설정은 config/ 한 곳에서 관리한다.

압축 해제 → 사내 모델 설정 → setup.ps1 → pi

```
PS> Expand-Archive .\cpp-csharp-lsp-lite-win-x64.zip -DestinationPath D:\
PS> cd D:\cpp-csharp-lsp-lite
PS> Copy-Item config\models.example.json config\models.json
# config\models.json의 baseUrl / model id를 사내 제공 값으로 편집
PS> setx /M CORP_LLM_API_KEY "<발급받은 API KEY>"   # 관리자 터미널
PS> Set-ExecutionPolicy -Scope Process Bypass
PS> .\setup.ps1
PS> cd D:\work\MyProject; pi   # 새 터미널
```

필드	설정 기준
`baseUrl` / `models[].id`	사내 API 안내에 명시된 주소와 모델 ID 사용
`apiKey`	`$CORP_LLM_API_KEY` 참조 유지 · 키는 시스템 변수로 등록
`contextWindow` / `maxTokens`	예제 숫자를 그대로 가정하지 않고 모델 지원 범위로 조정

실행 성공과 프로젝트 분석 성공을 따로 확인

```
# 아무것도 바꾸지 않고 배포 파일 확인
.\setup.ps1 -Check
```

증상	먼저 확인할 항목
`pi`를 찾을 수 없음	새 터미널에서 실행했는지 · setup.ps1 실행 여부 · 시스템 Path
모델 연결 실패	사내 API 접근 · 키 · provider/model · API 호환 설정
헤더·타입·패키지 오류	C++ 컴파일 옵션·SDK / C# solution·SDK·기존 NuGet 환경
진단 pending / 서버 실패	서버 경로 · 프로젝트 루트 · 초기화·진단 제한 시간
서브에이전트 실행 실패	사내 모델 ID · foreground(async: false) · Windows 지원 범위

실제 프로젝트에서도 정의 탐색 → 오류 유발 → 수정 후 진단 변화 → 빌드·테스트를 확인한다.

부록 D · 근거와 문서

설명은 공식 규약과 현재 구현을 기준으로

주제	근거 / 상세 문서
LSP 정의·기능	Microsoft · Language Server Protocol
C++ 분석 맥락	clangd · Compile commands
pi 실행·확장	동봉 pi 0.87.1 extension 문서 · LSP 연결 문서
확장 지원 범위	pi-lens · pi-subagents standalone
배포·재현	README · 런타임 잠금 파일 · 확장 잠금 파일
검증·라이선스	Windows CI · tests/ · third_party

기준: 2026-09-29, 구현 커밋 97b5071. 업무 효과 비교는 평가 계획이며 실측 결과가 아니다.